Inés Larrañaga Fernández de Pinedo	Egibide
	SGE

Table of Contents

Autenticación
- Introducción a la Autenticación en Django
  - Características Principales de `django.contrib.auth`:
- Práctica
  - Paso 1: Habilitar el sistema de autenticación de Django
  - Paso 2: Modelo de `User` personalizado
  - Paso 3: Sistema de login con email (usado FBV y templates)
  - Paso 4: Registro
  - Paso 5: Logout
  - Paso 6: Restringir acceso a vistas con `login_required`

Autorización
- Crear grupos y asignarles usuarios
- Restringir vistas por grupos
- Modificar login y añadirlo a un grupo

Autenticación

En este apartado, implementaremos autenticación y autorización en nuestro proyecto Django con vistas basadas en funciones (FBV). Esto nos permitirá gestionar usuarios, restringir el acceso a ciertas partes de la aplicación y definir permisos según roles.

Como tal, la **autenticación** es el proceso de verificar la identidad de un usuario, asegurando que quien intenta acceder a la aplicación es quien dice ser. En Django, esto se gestiona con el sistema `django.contrib.auth`, que permite el inicio y cierre de sesión, así como la gestión de credenciales.

Introducción a la Autenticación en Django

Django proporciona un sistema de autenticación robusto a través de la aplicación `django.contrib.auth`, que gestiona usuarios, autenticación, permisos y grupos.

Características Principales de `django.contrib.auth`:

- Autenticación de usuarios con `username` y `password` por defecto.

- Sistema de permisos y grupos para control de acceso.
- Funciones y decoradores como `login_required` para proteger vistas.
- Soporte para autenticación con sesiones y tokens.
- Modelos personalizables mediante `AbstractUser` y `AbstractBaseUser`.

Por defecto, Django usa un modelo de usuario que requiere un `username`. Sin embargo, en este proyecto vamos a personalizarlo para usar `email` como identificador.

Práctica

Paso 1: Habilitar el sistema de autenticación de Django

Como hemos dicho, Django ya incluye un sistema de autenticación integrado en `django.contrib.auth`, por lo que solo debemos asegurarnos de que esté habilitado en `settings.py`.

En `INSTALLED_APPS` de `settings.py`, debe estar:

```
INSTALLED_APPS = [
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'autenticacion',
    'empresa'
]
```

Y revisa más abajo que `MIDDLEWARE` incluya:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]
```

Paso 2: Modelo de `User` personalizado

Ahora vamos a crear una app nueva con el objetivo de separar del resto de aplicaciones, la funcionalidad de autenticacion que es transversal y no específica de ningún componente, sino más bien de todos:

```
python manage.py startapp autenticacion
```

Django usa `username` por defecto para la autenticación, pero lo vamos a cambiar para que los usuarios inicien sesión con su email. Para ello, creamos un modelo de usuario personalizado en `autenticacion/models.py`:

```
class CustomUserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError("El email es obligatorio")
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(email, password, **extra_fields)

class CustomUser(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True)
    name = models.CharField(max_length=255)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)

    objects = CustomUserManager()

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['name']

    def __str__(self):
        return self.email
```

- `CustomUser`:
    - `PermissionsMixin`: Agrega funcionalidades de permisos y grupos al usuario.

- Extiende de `AbstractBaseUser`: Proporciona las funcionalidades básicas de autenticación sin campos predefinidos como `username`. De forma que a partir de ella, añadimos los atributos de la clase que podamos necesitar en nuestro proyecto:
  - `email`: Campo único que sustituye al `username`.
  - `name`: Campo de nombre.
  - `is_active`: Indica si el usuario está activo.
  - `is_staff`: Indica si el usuario tiene acceso al panel de administración.
- `USERNAME_FIELD`: Especifica que el `email` será el campo de autenticación principal.
- `REQUIRED_FIELDS`: Lista de campos adicionales requeridos al crear un usuario con `createsuperuser`.

- `CustomUserManager`:
  - `BaseUserManager`: Se usa para definir la creación de usuarios y superusuarios de manera personalizada.
    - `create_user()`: Crea un usuario normal asegurando que el email es obligatorio y guardando la contraseña de forma segura.
    - `create_superuser()`: Crea un usuario administrador con permisos de superusuario y staff.

Luego, en `settings.py`, indicamos que este es nuestro modelo de usuario:

```
AUTH_USER_MODEL = 'autenticacion.CustomUser'
```

Aplicamos las migraciones:

```
python manage.py makemigrations autenticacion
python manage.py migrate
```

Paso 3: Sistema de login con email (usado FBV y templates)

Para permitir el inicio de sesión con email, creamos un nuevo formulario en `autenticacion/forms.py`:

```python
class LoginForm(forms.Form):
    email = forms.EmailField()
    password = forms.CharField(widget=forms.PasswordInput)
    def clean(self):
        email = self.cleaned_data.get('email')
        password = self.cleaned_data.get('password')
        user = authenticate(email=email, password=password)
        if not user:
            raise forms.ValidationError("Credenciales incorrectas")
        return self.cleaned_data
```

Ahora creamos la vista para el login en `autenticacion/views.py`:

```python
from django.contrib.auth import login, authenticate
from django.shortcuts import render, redirect
from autenticacion.forms import LoginForm

def login_view(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']
            user = authenticate(email=email, password=password)
            if user:
                login(request, user)
                return redirect('listar_empleados')
    else:
        form = LoginForm()
    return render(request, 'autenticacion/login.html', {'form': form})
```

Ahora creamos la plantilla en `templates/autenticacion/login.html`:

```
{% if messages %}
  <ul class="messages">
    {% for message in messages %}
      <li{% if message.tags %} class="{{ message.tags }}"{% endif %}>
        {{ message }}
      </li>
    {% endfor %}
  </ul>
{% endif %}
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Iniciar sesión</button>
</form>
```

Y registramos la URL en `autenticacion/urls.py` y recordad enlazadlo en el `urls.py` del proyecto:

```
from .views import login_view
from django.urls import path

urlpatterns = [
     path('login/', login_view, name='login'),
]
```

Paso 4: Registro

En el fichero `forms.py` de la app de `autenticacion` añadimos un nuevo formulario `RegisterForm` con los campos que vamos a necesitar para crear un formulario:

```
class RegisterForm(forms.Form):
    email = forms.EmailField()
    password = forms.CharField(widget=forms.PasswordInput)
    name = forms.CharField(max_length=255)
    def clean(self):
        email = self.cleaned_data.get('email')
        password = self.cleaned_data.get('password')
        name = self.cleaned_data.get('name')
        return self.cleaned_data
```

Después dentro de `autenticacion/views.py` añadimos el método `registrar`:

```
def registrar(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']
            name = form.cleaned_data['name']
            if CustomUser.objects.filter(email=email).exists():
                messages.error(request, 'El email ya está registrado.')
                return render(request, 'autenticacion/registro.html', {'form':
form})
            #grupo = Group.objects.get(name="Empleado")
            user = CustomUser.objects.create_user(email=email,
password=password, name=name)
            #user.groups.add(grupo)
            messages.success(request, 'Usuario registrado exitosamente.')
            return redirect('login')
        # Asigna automáticamente al grupo "Empleado"
        else:
            return render(request, 'autenticacion/registro.html', {'form':
form})
    form = RegisterForm()
    return render(request, 'autenticacion/registro.html', {'form': form})
```

Creamos el template correspondiente en `templates/autenticacion/registro.html`:

```
{% if messages %}
  <ul class="messages">
    {% for message in messages %}
      <li{% if message.tags %} class="{{ message.tags }}"{% endif %}>
        {{ message }}
      </li>
    {% endfor %}
  </ul>
{% endif %}
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Registrar</button>
</form>
```

Y finalmente añadimos la nueva ruta a un nuevo fichero creado dentro de `autenticacion/urls.py`:

```
from .views import registrar
from django.urls import path

urlpatterns = [
    path('registro/', registrar, name='registro'),
    ...
]
```

Y lo probamos convenientemente, comprobando después mediante login que funciona correctamente.

Paso 5: Logout

En `autenticacion/views.py` añadimos el método de logout:

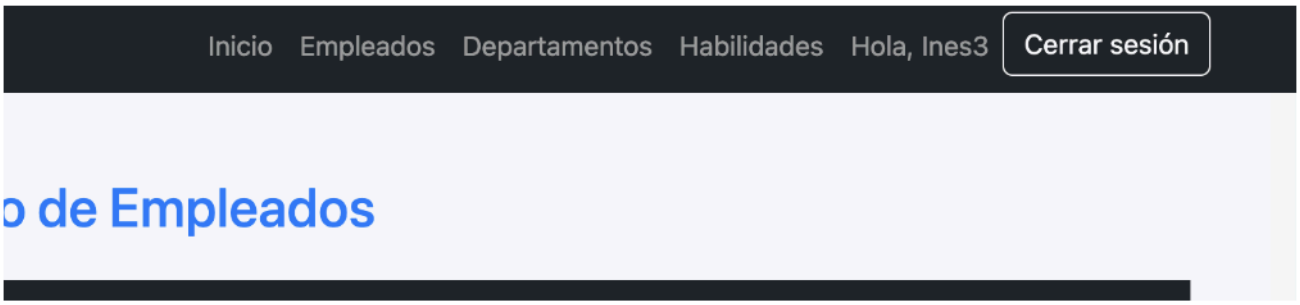
```
def logout(request):
     django.contrib.auth.logout(request)
     return redirect('login')
```

Añadimos la nueva ruta a `auth/urls.py`:

```
urlpatterns = [
    ...
    path('logout/', logout, name='logout'),
]
```

Y lo probamos convenientemente.

> Por tu cuenta, en caso de que el usuario esté logueado haz que se vea en la página de listado de empleados y pon un enlace para dar la posibilidad al usuario de cerrar sesión.



Paso 6: Restringir acceso a vistas con `login_required`

Puede resultar útil indicar a cuáles de las vistas que tenemos disponibles en nuestra aplicación se requiere un acceso autenticado. Eso lo podemos restringir usando el [decorador](Using the Django authentication system | Django documentation | Django) `@login_required`.

Por ejemplo, si vamos a `empresas/views.py`:

```
from django.contrib.auth.decorators import login_required
...
@login_required
def listar_empleados(request):
    empleados = Empleado.objects.all()
    # Configurar la paginación
    ...
```

Para que `@login_required` redirija a la página de login en caso de usuario no autenticado, debemos añadir en `settings.py`:

```
LOGIN_URL = reverse_lazy('login')
```

A veces resulta necesario, restringir el acceso de una forma más global, e ir de método en método añadiendo el decorador de login parece que no tenga demasiado sentido ni es muy mantenible ya que es fácil equivocarse.

En ese caso, cuando por ejemplo queremos cerrar el sitio de forma global exceptuando algunas acciones que no requieren autenticación, la forma de hacerlo sería mediante un `middleware`. Se puede implementar un **middleware personalizado** que verifique en cada request si el usuario está autenticado, y de no ser así, redirigirlo a la página de inicio de sesión.

Crea un fichero `autenticacion/middlewares.py` y define el siguiente middleware:

```python
from django.shortcuts import redirect
from django.urls import reverse

class LoginRequiredMiddleware:
    """
    Middleware que obliga a que el usuario esté autenticado para acceder a
cualquier página,
    exceptuando aquellas definidas en la lista de URLs permitidas.
    """
    # URLs a las que se permite el acceso sin autenticación
    ALLOWED_URLS = [
        reverse('login'),
        reverse('registro'),
        # Agrega otras URLs públicas o estáticas, por ejemplo:
        # reverse('contaco'), si existiera
    ]

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Si el usuario no está autenticado y la URL no es una de las
permitidas, redirige al login
        if not request.user.is_authenticated:
            # Para evitar bucles, se puede verificar que la URL solicitada no
esté en ALLOWED_URLS
            if request.path not in self.ALLOWED_URLS:
                return redirect('login')
        response = self.get_response(request)
        return response
```

Ahora, hay que activar el middleware en el proyecto dentro del `settings.py`:

```python
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'autenticacion.middlewares.LoginRequiredMiddleware',#Añadir el middleware
después del AuthenticacionMiddleware
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Autorización

De forma general, la **autorización** es el proceso de determinar qué acciones puede realizar un usuario una vez autenticado. Esto se logra mediante permisos y grupos de usuarios, lo que permite restringir el acceso a ciertas vistas o funcionalidades según el rol asignado.

Para ello, podemos asignar permisos y roles con el modelo `Group` de Django.

Crear grupos y asignarles usuarios

Este punto lo vamos a hacer mediante una migración personalizada, para que existan dichos grupos dentro del histórico de nuestro modelo en BBDD y se migre al pasar a cualquier entorno nuevo.

Para ello primero vamos a crear un fichero de migración vacía:

```
python manage.py makemigrations empresa --empty --name crear_grupos
```

Y ahora edita el fichero `empresa/migrations/XXXX_crear_grupos.py`:

```
from django.contrib.auth.models import Group
from django.db import migrations

def crear_grupos(apps, schema_editor):
    Group.objects.get_or_create(name="Gerente")
    Group.objects.get_or_create(name="Empleado")

class Migration(migrations.Migration):

    dependencies = [
        ('empresa', '0002_empleado_imagen'),
    ]

    operations = [
        migrations.RunPython(crear_grupos),
    ]
```

y migramos:

```
python manage.py migrate
```

Restringir vistas por grupos

Podemos usar el decorador `@user_passes_test` para restringir vistas según el grupo del usuario.

En `autenticacion/utils.py`:

```
from django.contrib.auth.models import Group

def es_gerente(user):
    return user.groups.filter(name='Gerente').exists()
```

Y la restricción lo podemos definir a nivel de ruta o dentro de views.py:

1. A nivel de ruta en `empresa/urls.py`:

   ```
   urlpatterns = [
   ...
       path('empleados/crear/', user_passes_test(es_gerente)(crear_empleado),
   name='crear_empleado'),
   ...
   ]
   ```

2. A nivel de `empresa/views.py`

```
@user_passes_test(es_gerente)
def crear_empleado(request):
    if request.method == 'POST' and request.FILES:
        form = EmpleadoFormulario(request.POST, request.FILES)
...
```

Modificar login y añadirlo a un grupo

Puede ser interesante que al registrar un usuario nuevo se añada a un grupo de forma automática. Vamos a suponer que queramos meterlos automáticamente como `Empleado`. Dentro de `autenticacion/views.py`:

```
        ...
        grupo = Group.objects.get(name="Empleado")
        user = CustomUser.objects.create_user(email=email, password=password,
name=name)
        user.groups.add(grupo)
        messages.success(request, 'Usuario registrado exitosamente.')
        return redirect('login')
        ...
```

Y lo comprobamos.